Unit 2

OOP using Python

- Handling exceptions,
- Exceptions as a control flow mechanism,
- Assertions, Abstract Data Types and Classes,
- Inheritance,
- Encapsulation and information hiding,
- Search Algorithms, Sorting Algorithms,
- Hashtables

Handling exceptions

- Error in Python can be of two types i.e. Syntax errors and Exceptions.
- Errors are problems in a program due to which the program will stop the execution.
- On the other hand, exceptions are raised when some internal events occur which change the normal flow of the program.

Different types of exceptions in python

1. **SyntaxError**: This exception is raised when the interpreter encounters a syntax error in the code, such as a misspelled keyword, a missing colon, or an unbalanced parenthesis.
2. **TypeError**: This exception is raised when an operation or function is applied to an object of the wrong type, such as adding a string to an integer.
3. **NameError**: This exception is raised when a variable or function name is not found in the current scope.
4. **IndexError**: This exception is raised when an index is out of range for a list, tuple, or other sequence types.

Different types of exceptions in python

5. **KeyError**: This exception is raised when a key is not found in a dictionary.
6. **ValueError**: This exception is raised when a function or method is called with an invalid argument or input, such as trying to convert a string to an integer when the string does not represent a valid integer.
7. **AttributeError**: This exception is raised when an attribute or method is not found on an object, such as trying to access a non-existent attribute of a class instance.

Different types of exceptions in python

8. **IOError**: This exception is raised when an I/O operation, such as reading or writing a file, fails due to an input/output error.
9. **ZeroDivisionError**: This exception is raised when an attempt is made to divide a number by zero.
10. **ImportError**: This exception is raised when an import statement fails to find or load a module.

- **Example:**
- Here in this code a s we are dividing the **‘marks’** by zero so a error will occur known as **‘ZeroDivisionError’**

```
marks = 10000
a = marks / 0
print(a)
```

Output:

```
Traceback (most recent call last):
  File "/home/f3ad05420ab851d4bd106ffb04229907.py", line 4, in <module>
    a=marks/0
ZeroDivisionError: division by zero
```

Try and Except Statement – Catching Exceptions

- Try and except statements are used to catch and handle exceptions in Python.
- Statements that can raise exceptions are kept inside the try clause and the statements that handle the exception are written inside except clause.

```
a = [1, 2, 3]
try:
        print ("Second element = %d" %(a[1]))
        print ("Fourth element = %d" %(a[3]))
except:
        print ("An error occurred")
```

- **Output:**

```
Second element = 2
An error occurred
```

Catching Specific Exception

```
def fun(a):
    if a < 4:
        b = a/(a-3)
    print("Value of b = ", b)
try:
    fun(3)
    fun(5)
except ZeroDivisionError:
    print("ZeroDivisionError Occurred and Handled")
except NameError:
    print("NameError Occurred and Handled")
```

Finally Keyword in Python

- Python provides a keyword finally, which is always executed after the try and except blocks.
- The final block always executes after the normal termination of the try block or after the try block terminates due to some exception
- **Syntax:**

```
try:
        # Some Code....
except:
        # optional block
        # Handling of exception (if required)
else:
        # execute if no exception
finally:
        # Some code .....(always executed)
```

Example:

```
try:
    k = 5/0
    print(k)
except ZeroDivisionError:
    print("Can't divide by zero")
finally:
    print('This is always executed')
```

Output:

Can't divide by zero

This is always executed

Advantages of Exception Handling:

- **Improved program reliability**: By handling exceptions properly, you can prevent your program from crashing or producing incorrect results due to unexpected errors or input.
- **Simplified error handling**: Exception handling allows you to separate error handling code from the main program logic, making it easier to read and maintain your code.
- **Cleaner code:** With exception handling, you can avoid using complex conditional statements to check for errors, leading to cleaner and more readable code.
- **Easier debugging**: When an exception is raised, the Python interpreter prints a traceback that shows the exact location where the exception occurred, making it easier to debug your code.

Disadvantages of Exception Handling

- **Performance overhead**: Exception handling can be slower than using conditional statements to check for errors, as the interpreter has to perform additional work to catch and handle the exception.
- **Increased code complexity**: Exception handling can make your code more complex, especially if you have to handle multiple types of exceptions or implement complex error handling logic.
- **Possible security risks**: Improperly handled exceptions can potentially reveal sensitive information or create security vulnerabilities in your code, so it’s important to handle exceptions carefully and avoid exposing too much information about your program

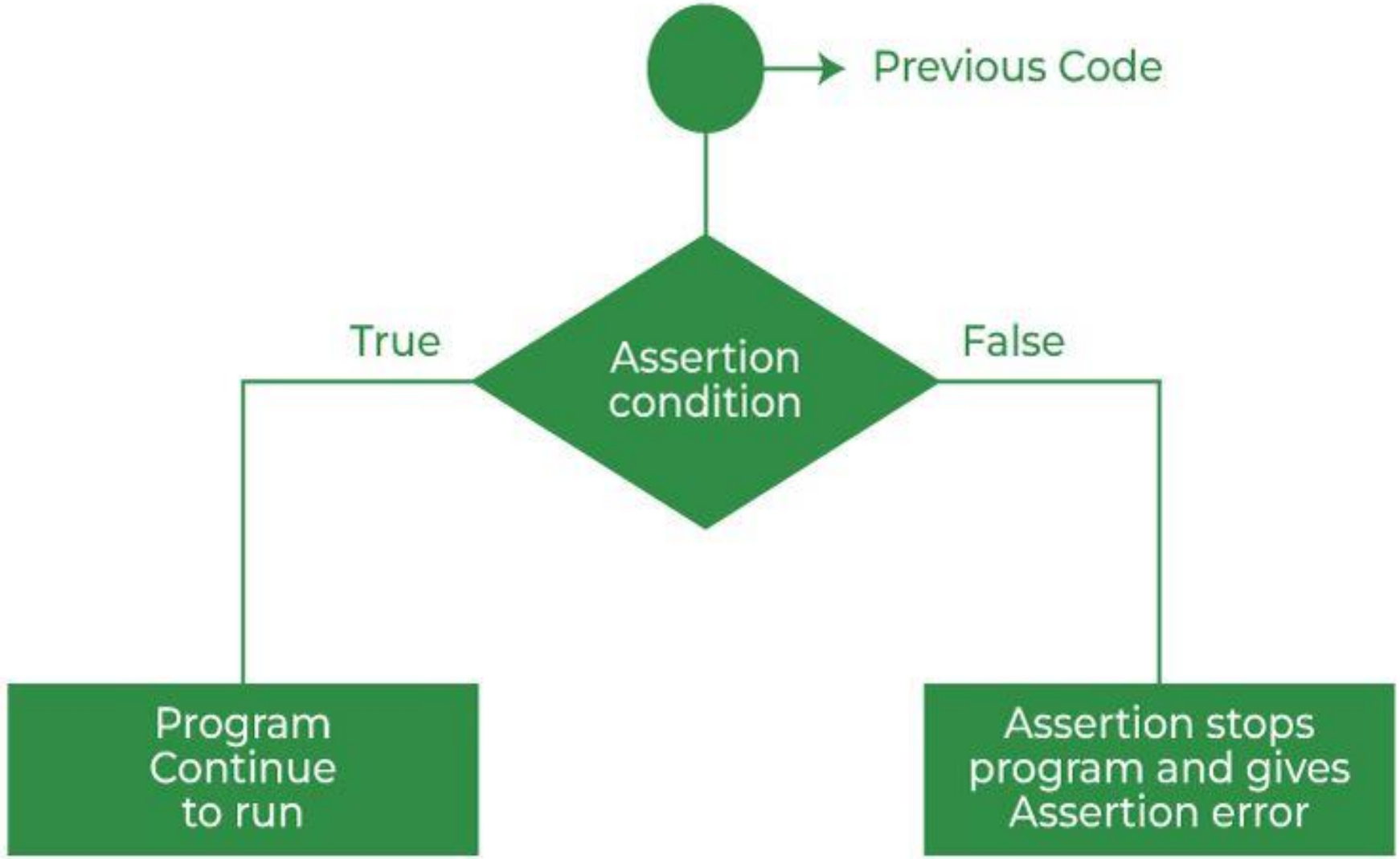
Assertions In Python

Assertions

- **Python Assertions** in any programming language are the debugging tools that help in the smooth flow of code.
- Assertions are mainly assumptions that a programmer knows or always wants to be true and hence puts them in code so that failure of these doesn’t allow the code to execute further.

Assert Keyword in Python

- In simpler terms, we can say that assertion is the boolean expression that checks if the statement is True or False.
- If the statement is true then it does nothing and continues the execution, but if the statement is False then it stops the execution of the program and throws an error.


Previous Code
Assertion condition
True
False
Program Continue to run
Assertion stops program and gives Assertion error

Python assert keyword Syntax

- **In Python, the assert keyword helps in achieving this task.**
- **This statement takes as input a boolean condition, which when returns true doesn’t do anything and continues the normal flow of execution, but if it is computed to be false, then it raises an AssertionError along with the optional message provided.**
- **Syntax : assert condition, error_message(optional)**

Example

```
a = 4
b = 2


print("The value of a / b is : ")
assert b != 0, "Zero Division Error"
print(a / b)
```

```
# Initializing variables
a = "hello"
b = 42

# Asserting the type of a variable
assert type(a) == str
assert type(b) == int

# Printing the values of a and b
print("a =", a)
print("b =", b)
```

object-oriented programming

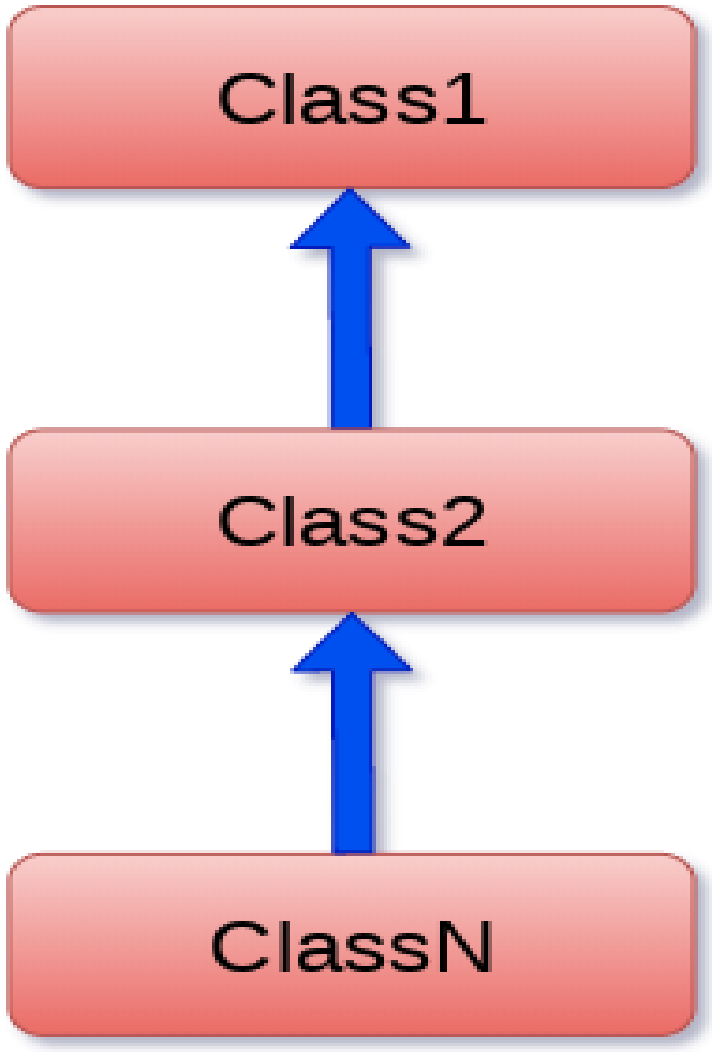
Python

```
Class BaseClass:
    {Body}
Class DerivedClass(BaseClass):
    {Body}
```

```
# A Python program to demonstrate inheritance
class Person(object):
  # Constructor
  def __init__(self, name, id):
    self.name = name
    self.id = id

  # To check if this person is an employee
  def Display(self):
    print(self.name, self.id)
# Driver code
emp = Person(“Yash", 10) # An Object of Person
emp.Display()
```

- **Multi-Level inheritance is possible in python like other object-oriented languages.**
- **Multi-level inheritance is archived when a derived class inherits another derived class.**
- **There is no limit on the number of levels up to which, the multi-level inheritance is archived in python.**


Class1
Class2
ClassN

Syntax

class class1:

<class-suite>

class class2(class1):

<class suite>

class class3(class2):

<class suite>

.

.

Example

```
class Animal:
    def speak(self):
        print("Animal Speaking")
#The child class Dog inherits the base class Animal
class Dog(Animal):
    def bark(self):
        print("dog barking")
#The child class Dogchild inherits another child class Dog
class DogChild(Dog):
    def eat(self):
        print("Eating bread...")
    d = DogChild()
    d.bark()
    d.speak()
    d.eat()
```

Output:

```
dog barking
Animal Speaking
Eating bread...
```

self method

- **The self parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.**
- **It must be provided as the extra parameter inside the method definition.**

Example:

```
class Details:
    name = “Yash"
    age = 25

    def desc(self):
        print("My name is", self.name, "and I'm", self.age, "years old.")

obj1 = Details()
obj1.desc()
```

__init__ method

- The __init__ method is used to initialize the object's state and contains statements that are executed at the time of object creation.
- Example:

```
class Details:
    def __init__(self, no, name):
        self.no = no
        self.name = name

obj1 = Details(1, “dog")
print(obj1.no, "belongs to the", obj1.name, "group.")
```

Multilevel Inheritance:

- Grandparent, parent, child…

```
class Machine:
    def start(self):
        print("Starting")

class Computer(Machine):
    def run_app(self):
        print("App running")
```

```
class SmartPhone(Computer):
    def call(self):
        print("Calling")

sp = SmartPhone()
sp.start()
sp.run_app()
sp.call()
```

Hierarchical Inheritance:

- One parent, multiple children

```
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Bow wow")

class Fish(Animal):
    def swim(self):
        print("Swimming")
```

```
dog = Dog()
dog.eat()
dog.bark()

fish = Fish()
fish.eat()
fish.swim()
```

Hybrid Inheritance: Combination of various types

```
class Animal:
    def eat(self):
        print("Eating")

class Mammal(Animal):
    def walk(self):
        print("Walking")

class Bird(Animal):
    def fly(self):
        print("Flying")
```

```
class Bat(Mammal, Bird):
    pass


bat = Bat()
bat.eat()
bat.walk()
bat.fly()
```

- **If the parent class has a constructor which takes arguments, then they must be passed through the child's constructor.**
- **For this purpose, super() function is used.**

```
class Parent:
    def __init__(self, num):
        print(num)

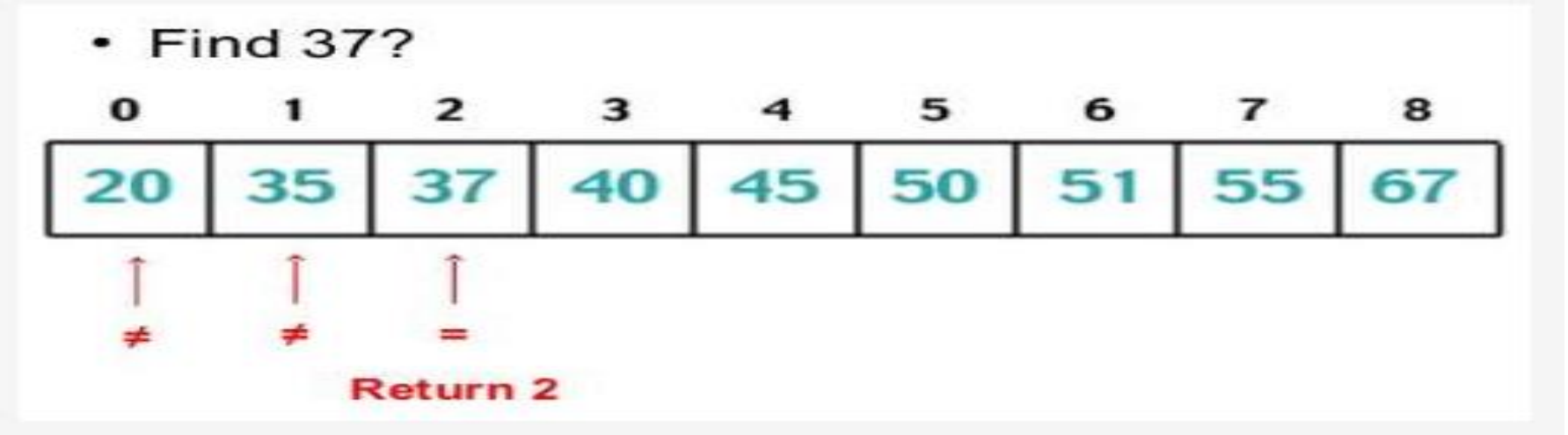
class Child(Parent):
    def __init__(self, num):
        return super().__init__(num)

child = Child(10)
```

SEARCHING

Linear Search:

- **Starting from the first, match each element with the search term and return the index wherever the match is found.**



```
def search(arr, N, x):

    for i in range(0, N):
        if (arr[i] == x):
            return i
    return -1

# Driver Code
arr = [2, 3, 4, 10, 40]
x = 4
N = len(arr)

# Function call
result = search(arr, N, x)
if(result == -1):
    print("Element is not present
in array")
else:
    print("Element is present at
index", result)
```

Binary Search

- **To perform binary search on a list, the list must be sorted.**
- **Steps:**

1. Find the midpoint and check if it matches with the search term, if it does then return its index.

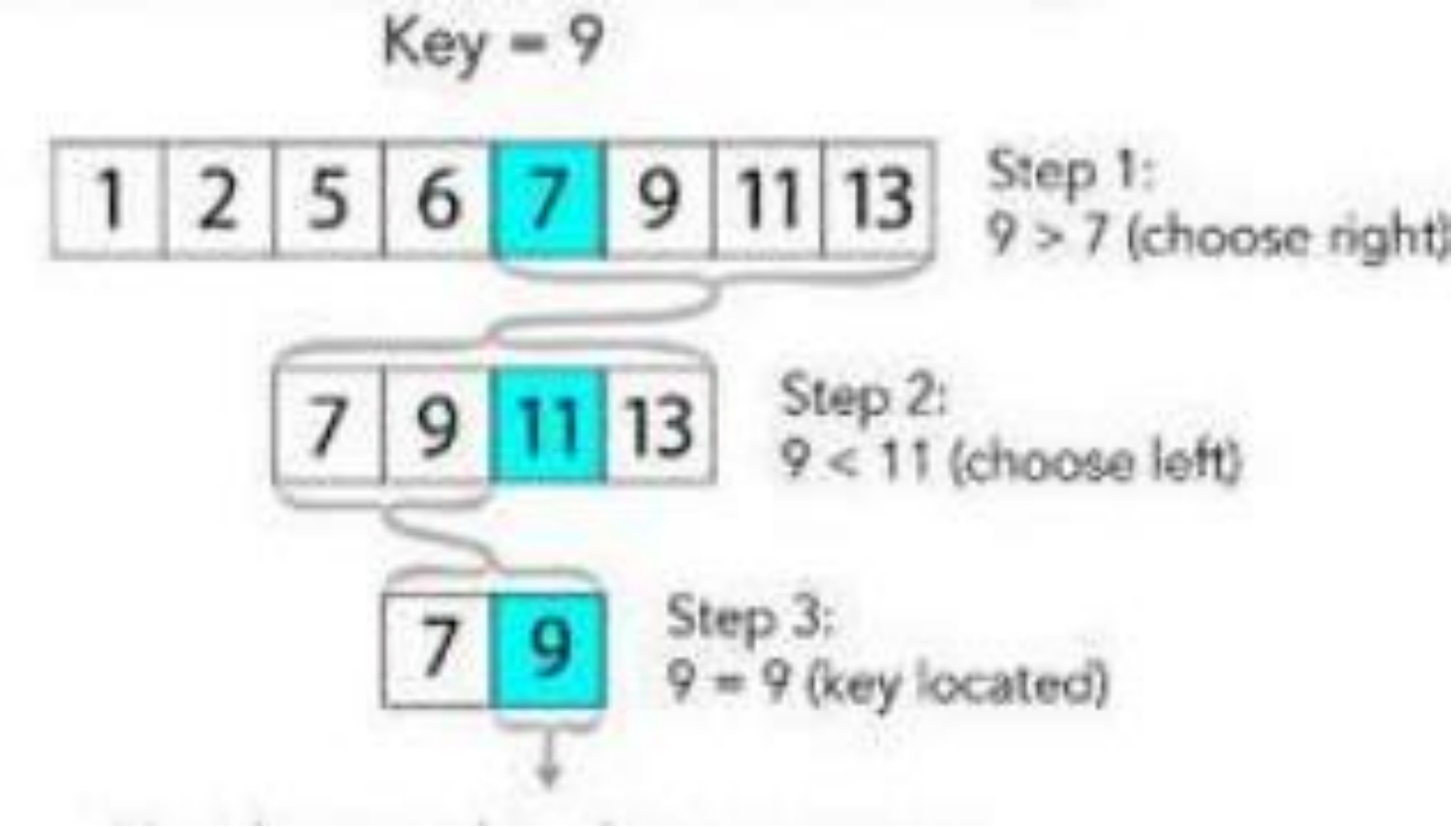
2. If the search term is less than the midpoint, repeat step 1 on the lesser-side of the midpoint.

3. If the search term is greater than the midpoint, repeat step 1 on the greater-side of the midpoint.

Conditions to apply Binary Search Algorithm in a Data Structure

- **The data structure must be sorted.**
- **Access to any element of the data structure should take constant time.**

Binary Search



```
def binarySearch(arr, low, high, x):

    while low <= high:

        mid = low + (high - low) // 2

        # Check if x is present at mid
        if arr[mid] == x:
            return mid

        # If x is greater, ignore left half
        elif arr[mid] < x:
            low = mid + 1

        # If x is smaller, ignore right half
        else:
            high = mid - 1

    # If we reach here, then the element
    # was not present
    return -1


arr = [2, 3, 4, 10, 40]
x = 10
# Function call
result = binarySearch(arr, 0, len(arr)-1, x)
if result != -1:
    print("Element is present at index",
result)
else:
    print("Element is not present in array")
```

SORTING

SORTING

- **Sorting is defined as an arrangement of data in a certain order.**
- **Sorting techniques are used to arrange data(mostly numerical) in an ascending or descending order.**
- **It is a method used for the representation of data in a more comprehensible format.**

Selection Sort

- **Each element of the list is checked with all the other elements on the right side one-by-one, starting from the first element.**
- **index has the smallest value is swapped with the current element which is being checked.**
- **This repeats until the end of the list.**

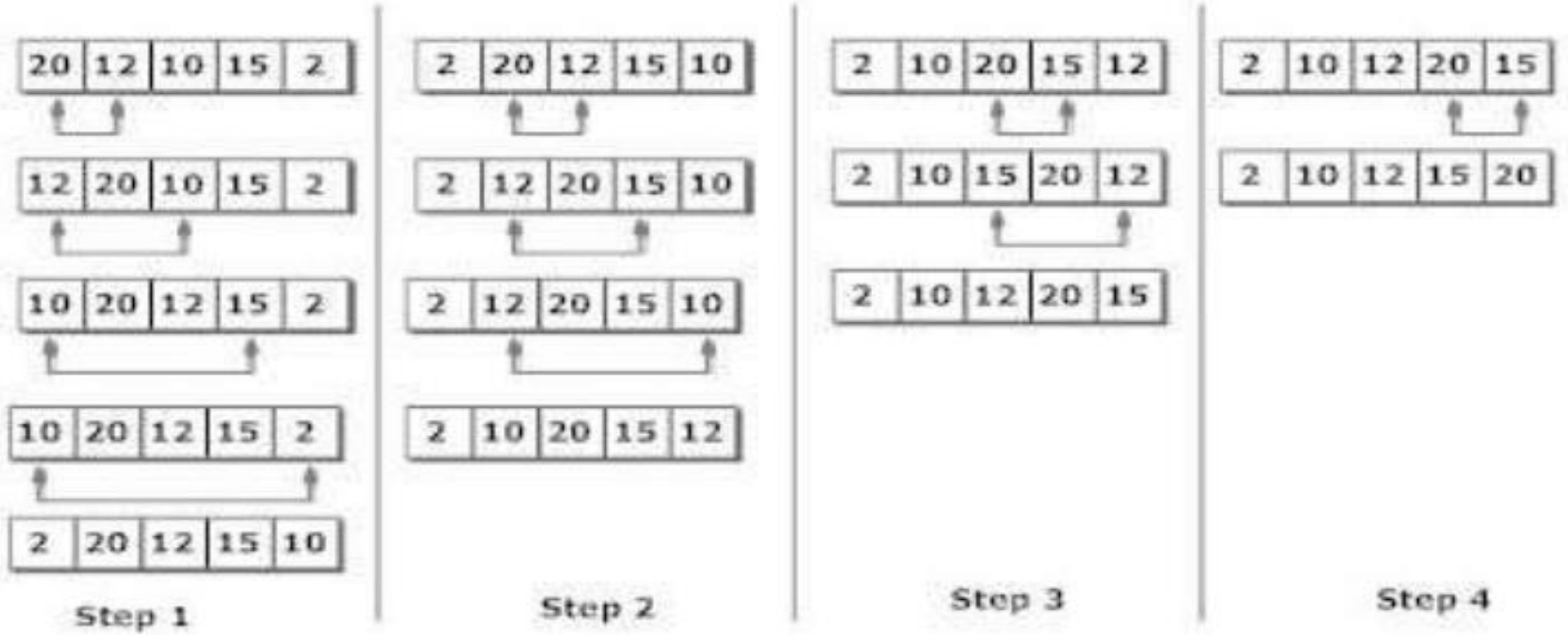


Figure: Selection Sort

```
# Selection Sort algorithm in Python
def selectionSort(array, size):

    for s in range(size):
        min_idx = s

        for i in range(s + 1, size):

            # For sorting in descending
order
            # for minimum element in
each loop
            if array[i] < array[min_idx]:
                min_idx = i


        # Arranging min at the correct
position
        (array[s], array[min_idx]) =
(array[min_idx], array[s])


# Driver code
data = [ 7, 2, 1, 6 ]
size = len(data)
selectionSort(data, size)


print('Sorted Array in Ascending
Order is :')
print(data)
```

Merge Sort

- **Merge Sort** is a **Divide and Conquer** algorithm
- **Find the midpoint and divide until individual elements are left.**
- **Starting with the first two, Merge the elements in a new list in ascending or descending order.**
- **Repeat first 2 steps for all parts**
- **Advantages of Merge Sort:**
- Stable sort (maintains relative order of equal elements).
- Guaranteed O(n log n) performance.
- Works well for large datasets or linked lists.
- **Disadvantages:**
- Requires additional space (not in-place).
- Slower for small lists compared to algorithms like insertion sort.

```
def merge_sort(arr):
    if len(arr) > 1:
        # Step 1: Divide
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        # Step 2: Recursively sort each half
        merge_sort(left_half)
        merge_sort(right_half)

        # Step 3: Merge the sorted halves
        i = j = k = 0

        # Compare and merge
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        # Remaining elements of left_half
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        # Remaining elements of right_half
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1

# Example usage
arr = [38, 27, 43, 3, 9, 82, 10]
print("Original array:", arr)
merge_sort(arr)
print("Sorted array:", arr)
```

Time and Space Complexity:

Case	Time Complexity
Best Case	O(n log n)
Average	O(n log n)
Worst Case	O(n log n)
Space	O(n)

Hashtables in Python

- A hashtable is a data structure that stores data in key-value pairs using a hash function.
- In Python, the built-in dictionary type (`dict`) acts as a hashtable.
- It allows fast access, insertion, deletion, and update operations using keys that are hashed to determine where the value is stored.
- **Key Characteristics:**
- Keys are unique and immutable (e.g., strings, numbers, tuples).
- Values can be of any data type and may be duplicated.
- Hashing is used internally for fast access.

```
student = {
    "name": "Yash",
    "age": 25,
    "course": "BCA"
}
# Printing the original hashtable
print("Original Hashtable:", student)
print("Hash Value is : ",hash("course"))

# Checking if a key exists
if "name" in student:
    print("Name exists in the hashtable.")
```

Output

```
Original Hashtable: {'name': 'Yash', 'age': 25, 'course': 'BCA'}
Hash Value is :  3657471069194180972
Name exists in the hashtable.
```